



THE CHIPS
TO SYSTEMS
CONFERENCE

SPONSORED BY:



HyPAS: A Hybrid Optimization Framework for Placement and Sizing Co-Optimization

Fangzhou Liu^{1,†} Wuqian Tang^{2,†} Bo-Ying Wang² An-Chieh Shen²
Han-Wen Tsao² Yuyang Ye¹ Yun Shao³ Chun-Yao Wang² Bei Yu¹

¹The Chinese University of Hong Kong ²National Tsing Hua University

³Giga Design Automation [†]Equal contribution



鸿芯微纳

Problem formulation

Input	Legal seed placement
Moves	Relocate cells; resize gates; insert buffer/inverter pairs
Variables	Cell positions and types (x, y, s)
Objective	Minimize $\alpha \text{TNS} + \beta \text{Power} + \gamma \text{HPWL}$
Penalties	Displacement and runtime

- Discrete sign-off flows^{1,2} are legal and timing-aware, but local and STA-heavy.
- Differentiable flows^{3,4} give global gradients, but relax discrete legality and timing boundaries.

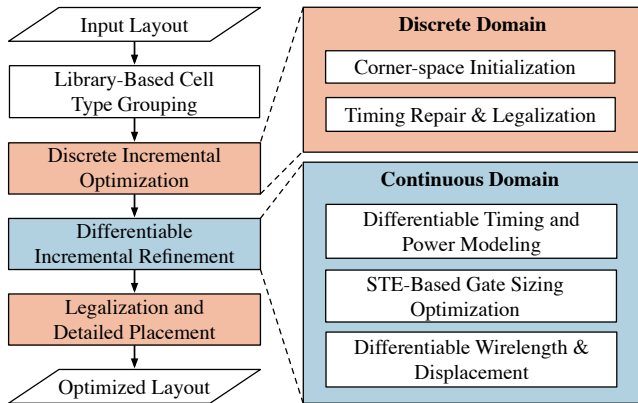
¹Ajayi and Blaauw, "OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Implementation Tool Chain," GOMACTech'19.

²Sze et al., "Path Based Buffer Insertion," DAC'05.

³Lin et al., "DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement," DAC'19.

⁴Du et al., "Fusion of Global Placement and Gate Sizing With Differentiable Optimization," ICCAD'24.

HyPAS Overview: Incremental Placement Needs Joint Decisions



The discrete stage gives a stable legal foundation; the differentiable stage explores broader PPA trade-offs.

Goal: legal drive/VT sizing candidates without hard placement violations.

Method

- Extract cell types and parameters from LIB/LEF.
- Build drive group $\mathcal{D}$ by matching ordered pins, output function, height, and width bucket g_w .
- Form $\mathcal{S} = \{(d, v) \mid d \in \mathcal{D}, v \in \{\text{R, L, SL}\}\}$ for VT delay-leakage trade-offs.

Example: resizing AND3x1_R

Cell type	Width	Group
AND3x1_R	0.324	$g_w = 0$
AND3x1_L	0.324	$g_w = 0$
AND3x2_R	0.378	$g_w = 0$
AND3x4_R	0.756	$g_w = 1$

Allowed: AND3x1_R, AND3x1_L, AND3x2_R.

Excluded: AND3x4_R: wider group.

Discrete Stage: Build a Timing-Clean Foundation



Input: a legal seed layout.

Output: a timing-clean, legal foundation for gradient refinement.

$$F_{\text{obj}}(\mathbf{x}, \mathbf{y}, \mathbf{s}) = \alpha \hat{F}_{\text{timing}}(\mathbf{x}, \mathbf{y}, \mathbf{s}) + \beta \hat{F}_{\text{power}}(\mathbf{x}, \mathbf{y}, \mathbf{s}) \\ + \gamma F_{\text{wire}}(\mathbf{x}, \mathbf{y}) + \lambda_{\text{disp}} F_{\text{disp}}(\mathbf{x}, \mathbf{y}) + \lambda_{\text{dens}} F_{\text{dens}}(\mathbf{x}, \mathbf{y})$$

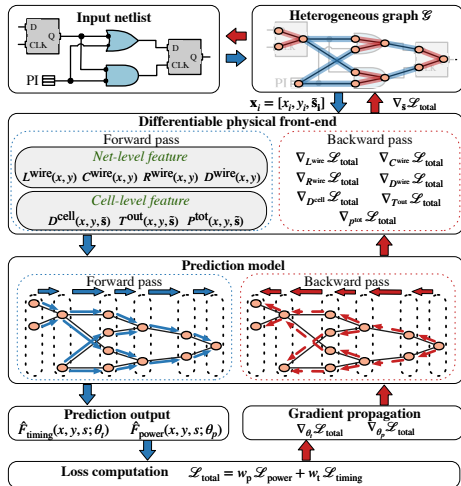
- Beyond DREAMPLACE placement/density gradients, HYPAS customizes all objective terms and bridges timing-power feedback to legal library choices.

Bridge 1: why timing/power model?

- Existing differentiable timing engines lack sizing/power support.
- GNN-based learned surrogates: $\sim 90\%$ accuracy, $\sim 1\text{s}$ inference.
- **Key insight:** gradient optimization needs direction, not perfection.

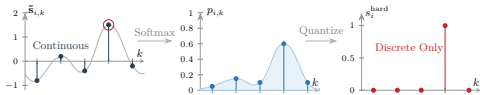
Bridge 2: why STE sizing?

- Gate sizing is inherently a **discrete optimization problem**.
- STE enables gradient-based optimization over discrete choices.
- Forward: real cell; backward: soft gradients.



- **Graph input:** netlist $\rightarrow$ pin graph with state $[x_i, y_i, \tilde{s}_i]$.
- **Physical front-end:** differentiable wire $L/C/R/D$ features; pretrained MLPs replace NLDM lookups for cell $D/T/P$.
- **GNN prediction:** message-passing networks with max aggregation (timing) and mean aggregation (power).
- **Gradient feedback:** $\mathcal{L}_{\text{total}}$ backpropagates to s-logsits; placement optimized by DREAMPLACE.

Forward: evaluate a real library cell



- Candidate set $\mathcal{S}_i$ comes from legal drive/VT library groups; learn s-logits and mask invalid entries.
- Soft relaxation maps logits to a differentiable size:

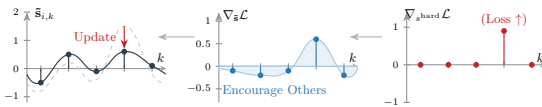
$$p_{i,k} = \text{softmax}(\tilde{s}_{i,k}/\tau),$$

$$s_i^{soft} = \sum_k p_{i,k} s_{i,k}.$$

- Forward evaluation chooses one real library cell:

$$s_i^{hard} = \mathcal{S}_{i, \arg \max_k p_{i,k}}.$$

Backward: update logits through relaxation



- Timing/power prediction evaluates s_i^{hard} with placement state, not a fractional cell.
- STE couples the hard forward value with the soft backward path:

$$s_i^{fwd} = s_i^{hard} - \text{detach}(s_i^{soft}) + s_i^{soft}.$$

- Prediction loss follows s_i^{soft} to update s-logits; deploy hard choices and legalize.

Setup

- Environment: 2025 ICCAD Contest Docker
- Benchmarks: Problem C (2K-170K cells)
- Technology: ASAP7 PDK
- Implementation: OPENROAD/DREAMPLACE
- Baselines: 1st/2nd/3rd teams, OPENROAD, DREAMPLACE 4.0

Overall Gains

- **24.95%** average TNS improvement
- **12.45%** average power improvement
- **3.21%** average HPWL improvement
- **40.62%** average total PPA improvement

Key Observations

- Best average total PPA among all baselines
- Practical runtime (except 2nd place & DREAMPLACE)
- Robust under different objective weights
- Ranks best on 9 of 13 benchmarks

Improvement ratio over the unoptimized seed placement ($\alpha = \beta = \gamma = 1$).

Benchmark	1st	2nd	3rd	OPENROAD	DREAMPLACE	HYPAS
des3	-0.68	13.05	-0.20	-23.73	6.30	11.77
des	13.42	13.36	5.37	-17.44	4.69	18.53
tv80s	-0.58	2.86	1.23	-6.93	3.44	7.14
aes	53.07	-27.32	-44.38	-32.86	-18.31	55.71
mc_top	-5.60	5.37	-1.93	-11.41	7.89	18.13
ac97_top	39.46	18.21	18.48	-12.79	23.99	60.37
ariane	80.17	82.57	85.72	53.35	-3.60	81.86
aes_cipher	48.08	40.54	-0.80	-15.69	13.35	48.91
pci	54.25	16.73	36.57	23.63	18.89	71.01
fpu	65.46	-4.72	48.75	46.89	-69.81	66.29
NVpartition	18.43	26.38	7.27	1.69	0.10	24.73
mempool	-6.08	21.50	-0.33	0.00	0.00	-0.79
netcard_fast	40.86	1.74	-0.02	0.00	-9.01	64.34
Average	30.79	16.17	11.98	0.36	-1.70	40.62

Key message: HYPAS has the best average total PPA, 40.62% vs. 30.79% for the strongest contest baseline, and ranks best on 9 of 13 benchmarks.

- HYPAS combines discrete sign-off repair with differentiable refinement for joint placement and sizing optimization.
- GNN-based timing/power model provides fast gradient feedback for multi-objective refinement.
- STE sizing enables gradient-based optimization over discrete library choices.
- ICCAD 2025 Problem C: 40.62% average PPA improvement with practical runtime.